

Hands-On: Understanding State and Async State Updates in React

Requirements

- Basic knowledge of React
- Text editor and web browser
- Node.js and npm installed on your computer

Instructions

1. Create a new React project using `create-react-app``.

```
npx create-react-app state-lab  
cd state-lab
```

2. Open the project folder in your text editor.
3. In the `src`` folder, create a new file named `StateComponent.js``.

```
import React, { Component } from "react";  
  
class StateComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0,  
    };  
  }  
  
  render() {  
    return (  
      <div>  
        <h1>Count: {this.state.count}</h1>  
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>  
          Increase Count  
        </button>  
      </div>  
    );  
  }  
}  
  
export default StateComponent;
```

4. In the `App.js`` file, import the `StateComponent`` and render it in the `render()`` method.

```
import React from "react";  
import StateComponent from "../StateComponent";  
  
function App() {  
  return (  
    <div className="App">
```

```

        <StateComponent />
      </div>
    );
  }

```

```
export default App;
```

5. Start the development server.

```
npm start
```

6. Open your browser and go to `http://localhost:3000/`.
7. You should see a "Count: 0" heading and an "Increase Count" button. Clicking the button should increase the count by one.

This means that the `StateComponent` is using state to store the count value and update it when the button is clicked.

8. Now, let's add an async state update to the `StateComponent`. Add the following code inside the `render()` method, right before the `return` statement:

```

setTimeout(() => {
  this.setState({ count: this.state.count + 1 });
}, 5000);

```

This means that after 5 seconds, the count will be increased by one automatically.

9. Save the file and refresh the browser. You should see the "Count: 0" heading and the "Increase Count" button. Wait for 5 seconds and you should see the count increase by one automatically.

This means that the async state update is working and the count is being increased after 5 seconds.

10. Finally, let's add a callback function to the `setState()` method to display the updated count in the console. Modify the `setState()` method in the `onClick` event of the button like this:

```

this.setState({ count: this.state.count + 1 }, () => {
  console.log("New count:", this.state.count);
});

```

This means that after the count is updated, the callback function will be called and the new count will be logged in the console.

11. Save the file and refresh the browser. You should see the "Count: 0" heading and the "Increase Count" button. Click the button a few times and you should see the new count values being logged in the console.

This means that the callback function is working and the new count value is being logged in the console after each update.

That's it! You've successfully completed the hands-on lab for defining state in components, updating state, and async state updates in React. You can now use these concepts to build more complex React applications with dynamic data that can be updated asynchronously.